

1. Skills Solve a Different Problem

A skill packages repeatable workflow knowledge, references, and sometimes scripts. This makes it suitable for tasks whose procedure matters but should not occupy every Codex turn. Plugins can bundle skills with tools, connectors, or other configuration for broader reuse.

A useful division is: prompt for a one-off constraint; AGENTS.md for durable project rules; skill for a repeatable procedure; plugin for a reusable bundle that needs distribution or integrated capabilities.

2. Kundel's Astra Warning

Dominik Kundel reports a counterintuitive launch experience: video-editing skills built around limitations of older models sometimes got in Astra's way. A more vanilla setup worked better. His recommendation is empirical: if a skill exists mainly because an older model could not perform a task directly, retry without it.

This does not imply that skills are obsolete. It implies that a skill must earn its complexity under the current model. A workflow encoding organizational requirements, dangerous boundaries, a specialized toolchain, or a reproducible evaluation remains valuable. A long sequence of compensatory micro-instructions may not.

3. The Skill-Retirement Test

For each skill, ask:

1. What failure was this built to prevent?
2. Does Astra still exhibit that failure on representative tasks?
3. Does the skill add organization-specific knowledge, or only generic prompting?
4. Does it improve measured success enough to justify the extra instruction surface?
5. Does it force a tool or sequence that blocks better routes now available?

Run an A/B comparison when the answer is unclear. The control is a clean or minimally configured Astra run. The treatment is the skill. Keep the task, initial state, and success criteria fixed.

4. Progressive Disclosure

A good skill should reveal detail when the workflow needs it rather than dumping a manual into the initial context. Its top-level description should make the trigger legible. The body should contain steps and boundary conditions. Supporting scripts and references should carry exact procedures or domain material.

This architecture reduces instruction noise. It also makes a skill testable as an independent unit: does the trigger fire on the intended task, does the procedure produce the required artifacts, and does it stop at the correct boundary?

5. Plugins and External Systems

Use a plugin or connector when the work depends on external systems, live workspace data, or actions that are better represented by structured tools than by browser scraping. Tool choice should reduce ambiguity. A typed GitHub operation communicates a repository action more precisely than a series of mouse clicks, while Computer Use remains valuable when no narrow integration exists.

6. Overfitting the Harness

A common customization failure is to encode one successful run into a permanent ritual. The resulting skill may demand a fixed sequence even when the next repository, model, or tool differs. This is procedural overfitting.

The cure is not vagueness. Preserve invariants and evidence requirements; relax incidental implementation steps. "Verify responsive layout at these widths" is portable. "Open exactly three screenshots in this order after every CSS edit" may be historical debris unless the workflow depends on it.

7. A Maintenance Rule

Every durable customization should have an owner, a reason to exist, and a cheap test of whether it still helps. Model upgrades are natural audit points. Astra's stronger instruction following makes this maintenance more valuable, not less.

References: OpenAI, Codex customization; OpenAI, GPT-6 Astra model guidance; Dominik Kundel, "Five things I learned from using Astra."

Independent Open Course

<https://github.com/dicnunz/codex-astra-course>

CODEX.00 | Codex with GPT-6 Astra

Handout 7: Kundel on Skills and Guidance

Fall 2026

License and source

Structural and pedagogical adaptation of MIT OpenCourseWare 24.00 Problems of Philosophy, Fall 2019, Prof. Miriam Schoenfield.

Original course and this adaptation are licensed CC BY-NC-SA 4.0.

This course is independently produced and is not affiliated with or endorsed by MIT or OpenAI.

Linked external readings and referenced product documentation retain their own copyrights and terms.

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

<https://ocw.mit.edu/courses/24-00-problems-of-philosophy-fall-2019/>